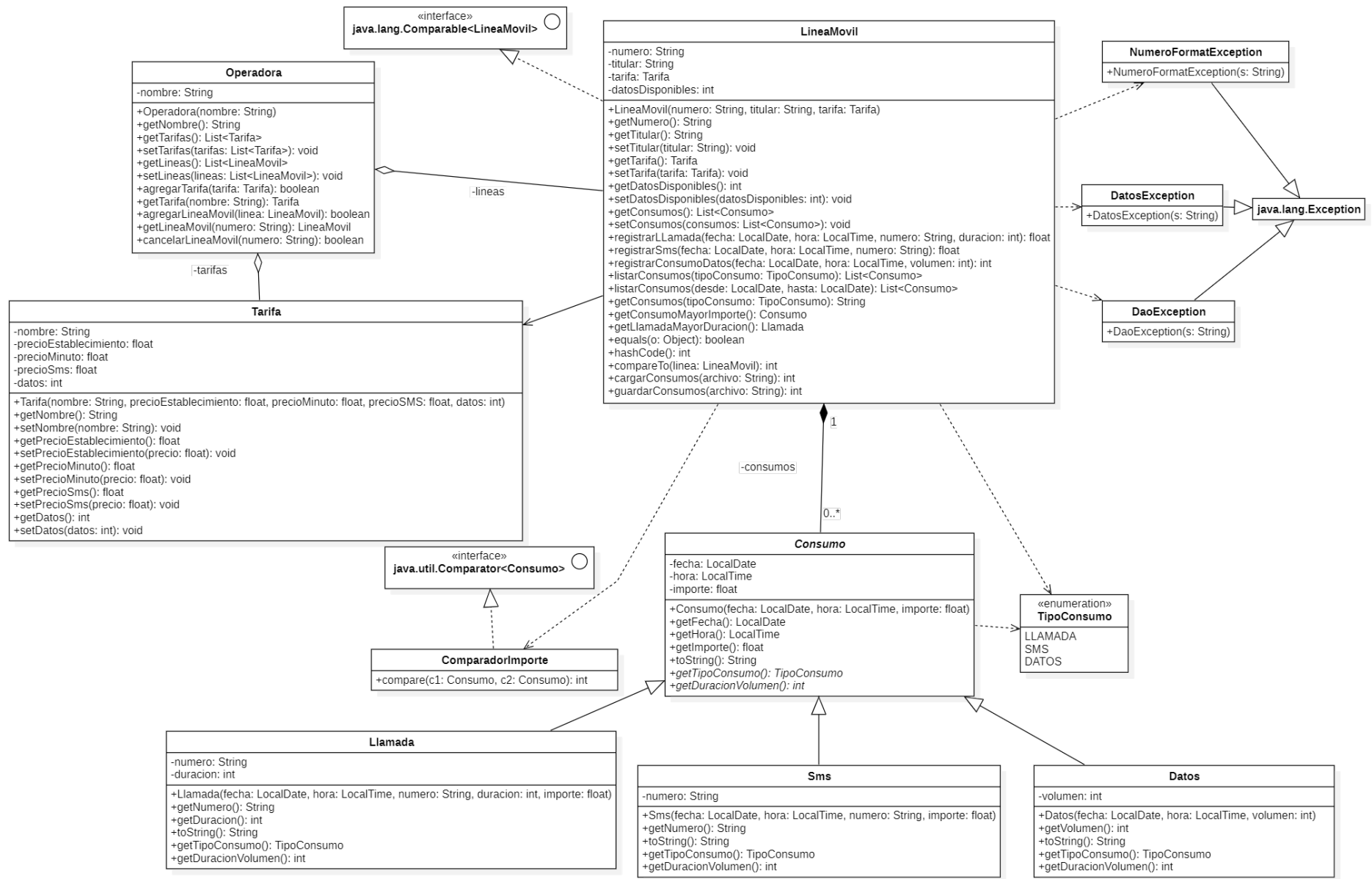


EXAMEN 2ª EVALUACIÓN - 14 de marzo de 2025

1. Se desea implementar una aplicación para la gestión de tarifas, líneas móviles y consumos de una operadora de telefonía móvil. Paquete: **es.sauces.examen2**



Desarrolla la enumeración y las clases que aparecen en el diagrama de clases. El proyecto se da parcialmente implementado.

Los errores que se produzcan, en los distintos métodos de las clases, los registraremos mediante Java Logging API, en el archivo registroExamen.log.
La configuración del administrador del registro se da implementada.

Se proporciona un archivo con información de consumos: *consumos.csv*.

Las clases deberán compilar sin errores y ajustarse a las especificaciones dadas.
Los métodos que tengan errores de ejecución, de lógica o de especificación se puntuarán con 0.

Entrega del examen:

Carpeta comprimida con nombre y apellidos que contenga este archivo y el proyecto.

Operadora

Clase que representa una operadora de telefonía móvil

Operadora(String nombre)

Inicializa el atributo nombre con el valor recibido y crea un **TreeSet** en el atributo *tarifas* y un **TreeMap** en el atributo *lineas*.

boolean agregarTarifa(Tarifa tarifa)

Añade la tarifa recibida al conjunto de tarifas. Devuelve **true** si la operación ha tenido éxito, **false** en caso contrario.

Tarifa getTarifa(String nombre)

Devuelve la tarifa cuyo nombre se corresponda con el nombre pasado como parámetro, **null** si no hay una tarifa con ese nombre.

boolean agregarLineaMovil(LineaMovil linea)

Añade la línea recibida al mapa de líneas. Devuelve **true** si la operación ha tenido éxito, **false** en caso contrario.

LineaMovil getLineaMovil(String numero)

Devuelve la línea cuyo número sea igual que el número pasado como parámetro, **null** si no hay ninguna línea con ese número.

boolean cancelarLineaMovil(String numero)

Elimina, del mapa de líneas, la línea móvil cuyo número sea igual que el número pasado como parámetro. Devuelve **true** si la operación ha tenido éxito, **false** en caso contrario.

LineaMovil

Clase que representa una línea móvil.

LineaMovil(String numero, String cliente, Tarifa tarifa)

Si el número no se ajusta al patrón (9 dígitos, el primero será 6 ó 7) lanza **NumberFormatException**. Comprobar mediante una expresión regular.

Inicializa los atributos *numero*, *cliente* y *tarifa* con los valores recibidos. Inicializa el atributo *datosDisponibles* al valor devuelto por el método *getDatos()* del atributo *tarifa* multiplicado por 1000 y en el atributo *consumos* crea un **LinkedList**.

float registrarLLlamada(LocalDate fecha, LocalTime hora, String numero, int duracion)

Si la duración es negativa lanza **IllegalArgumentException**.

Crea un objeto de la clase **Llamada** y lo añade al listado de consumos. La duración de la llamada la recibe en segundos.

Para calcular el importe: **importe=precioEstablecimiento+Math.ceilDiv(duracion,60)*precioMinuto**

Devuelve el importe de la llamada.

float registrarSms(LocalDate fecha, LocalTime hora, String numero)

Crea un objeto de la clase **Sms** y lo añade al listado de consumos.

Para calcular el importe: **importe=precioSMS**

Devuelve el importe del mensaje.

int registrarConsumoDatos(LocalDate fecha, LocalTime hora, int volumen)

Si el volumen es negativo o mayor al valor del atributo datosDisponibles lanza **DatosException**.

Crea un objeto de la clase **Datos** y lo añade al listado de consumos.

El volumen lo recibe en kB.

Cambia el valor del atributo datosDisponibles: **datosDisponibles=datosDisponibles - volumen**

Devuelve el valor del atributo datosDisponibles.

List<Consumo> listarConsumos(TipoConsumo tipoConsumo)

Devuelve el listado de consumos del tipo recibido como parámetro.

List<Consumo> listarConsumos(LocalDate desde, LocalDate hasta)

Devuelve el listado de consumos registrados entre dos fechas dadas.

String getConsumos(TipoConsumo tipoConsumo)

Cuenta el número de consumos del tipo recibido como parámetro que hay en el listado de consumos y acumula el valor devuelto por el método **getDuracionVolumen()** de cada uno de los consumos del tipo recibido.

Devuelve una cadena compuesta por el número de consumos y por el acumulador, separados por comas. Utiliza **ListIterator** para recorrer el listado de consumos.

Ejemplos:

TipoMovimiento.LLAMADA	→	"32,7870"
TipoMovimiento.SMS	→	"38,38"
TipoMovimiento.DATOS	→	"31,268987"

Consumo getConsumoMayorImporte()

Devuelve el consumo cuyo importe sea mayor.

Llamada getLlamadaMayorDuracion()

Devuelve la llamada de mayor duración.

boolean equals(Object o)

Compara el objeto recibido con esta línea.

Devuelve true si el objeto recibido es una instancia de la clase **LineaMovil** y su atributo numero coincide con el atributo numero de esta línea. En caso contrario devuelve **false**.

int compareTo(LineaMovil linea)

Compara dos líneas. La comparación se basa en el atributo numero.

Devuelve 0 si los atributos numero de las dos líneas son iguales; un valor menor que 0 si el atributo numero de esta línea es menor que el atributo numero de la línea recibida como argumento; un valor mayor que 0 si el atributo numero de esta línea es mayor que el atributo numero de la línea recibida como argumento.

int cargarConsumos(String archivo)

Lee los consumos almacenados en el archivo recibido como parámetro y los añade al **List consumos**.

Cada línea del archivo almacena un consumo en formato CSV.

Contenido de cada línea:

Llamada: LLAMADA, fecha, hora, importe, número, duración (LLAMADA, 2025-03-22, 07:27, 0.36, 637590504, 60)

Sms: SMS, fecha, hora, importe, número (SMS, 2025-03-22, 06:07, 0.18, 721347056)

Datos: DATOS, fecha, hora, importe, volumen (DATOS, 2025-03-16, 12:02, 0.0, 7148)

Devuelve el número de los consumos leídos.

Cualquier excepción debe capturarse y lanzar **DaoException**.

int guardarConsumos(String archivo)

Almacena los consumos en el archivo recibido como parámetro. Se almacenará un consumo en cada línea. Contenido de cada línea: cadena devuelta por el método **toString()** de la clase **Consumo**.

Devuelve el número de consumos almacenados.

Cualquier excepción debe capturarse y lanzar **DaoException**.

Ejemplos:

DATOS, 2025-01-01, 09:51, 0.0, 7182

LLAMADA, 2025-01-02, 07:09, 0.78, 623260879, 480

SMS, 2025-01-03, 03:46, 0.18, 756189243

Consumo Clase que representa un consumo en la línea móvil.

Consumo(LocalDate fecha, LocalTime hora, float importe)

Inicializa los atributos con los datos recibidos.

String toString()

Devuelve una cadena compuesta por tipo de consumo, fecha, hora e importe separados por comas.

Ejemplo: "LLAMADA, 2025-03-22, 07:27, 0.36"

TipoConsumo getTipoConsumo() Método abstracto.

int getDuracionVolumen() Método abstracto.

Llamada Clase que representa una llamada de teléfono.

Llamada(LocalDate fecha, LocalTime hora, String numero, int duracion, float importe)

Inicializa los atributos con los valores recibidos.

String toString()

Devuelve una cadena compuesta por tipo de consumo, fecha, hora, importe, número y duración separados por comas.

Ejemplo: "LLAMADA,2025-03-22,07:27,0.36,637590504,60"

TipoConsumo getTipoConsumo()

Devuelve **TipoConsumo.LLAMADA**

int getDuracionVolumen()

Devuelve la duración de la llamada.

Sms Clase que representa un mensaje de texto.

Sms(LocalDate fecha, LocalTime hora, String numero, float importe)

Inicializa los atributos con los valores recibidos.

String toString()

Devuelve una cadena compuesta por tipo de consumo, fecha, hora, importe y número separados por comas.

Ejemplo: "SMS,2025-03-22,06:07,0.18,721347056"

TipoMovimiento getTipo()

Devuelve **TipoConsumo.SMS**

int getDuracionVolumen()

Devuelve 1.

Datos Clase que representa un consumo de datos.

Datos(LocalDate fecha, LocalTime hora, int volumen)

Inicializa los atributos con los valores recibidos. El importe será 0.

String toString()

Devuelve una cadena compuesta por tipo de consumo, fecha, hora, importe y volumen separados por comas.

Ejemplo: "DATOS,2025-03-16,12:02,0.0,7148"

TipoConsumo getTipoConsumo()

Devuelve **TipoConsumo.DATOS**

int getDuracionVolumen()

Devuelve el volumen de datos consumidos.

CALIFICACIÓN

	Correcto. Se ajusta a especificaciones.	Correcto pero con problemas de claridad, legibilidad o eficiencia.	Sin realizar. No se ajusta a especificaciones. Errores de lógica. Errores de ejecución.
Operadora	1	0,5	0
TipoConsumo	0,2	0,1	0
Consumo	0,2	0,1	0
Llamada	0,2	0,1	0
Sms	0,2	0,1	0
Datos	0,2	0,1	0
ComparadorImporte	0,2	0,1	0
DaoException, NumeroFormatException, DatosException	0,2	0,1	0
LineaMovil (constructor)	0,2	0,1	0
LineaMovil (métodos equals y compareTo)	0,2	0,1	0
LineaMovil (métodos registrarLlamada, registrarSms, registrarConsumoDatos)	0,6	0,3	0
LineaMovil (método listarConsumos(TipoConsumo tipoConsumo))	0,6	0,3	0
LineaMovil (método listarConsumos(LocalDate desde, LocalDate hasta))	1	0,5	0
LineaMovil (método getConsumos)	1	0,5	0
LineaMovil (método getConsumoMayorImporte)	0,4	0,2	0
LineaMovil (método getLlamadaMayorDuracion)	1	0,5	0
LineaMovil (método guardarConsumos)	0,4	0,2	0
LineaMovil (método cargarConsumos)	1	0,5	0
Registro de errores	1	0,5	0
Excepciones	0,2	0,1	0
TOTAL	10		